

Nombre: Frank Cristian Pardo Cuy
Codigo: 21200065

Fecha: 20-06-26

I.

I.1: El pattern repositorio como SSD+

Se considera como unica fuente de verdad porque ~~centraliza~~ centraliza el acceso y la gestion de los datos de la aplicacion.

Ademas la UI no consulta ni a la base de datos local ni a la API remota de forma directa solo con el repositorio.

Funciona como "Fachada" porque este no deja ver lo complejo que es internamente la fuente de datos.

El repositorio coordina de forma que:

UI → View model → Repository → Room / Retrofit

I.2: Corrutinas y Structured Concurrency

View Model Scope

El ~~View Model~~ es un scope de corrutinas ~~de~~ que esta relacionado al ciclo de vida del View Model y sirve para evitar fugas de memoria ya que cuando el View Model se cancela o destruye las corrutinas iniciadas en el scope se cancelan de forma automatica. Por eso cuando el usuario cierra ~~la pantalla~~ la pantalla antes de que la peticion de red que realice envie respuesta, las corrutinas se cancelan y la peticion ya no deberian actualizar la UI.

I.3: Fljos Reactivos (Cold vs Hot)

~~Fljos~~
Flow

Cold: no emite datos hasta que un collect, recolecta informacion o data ademas que puede volver a ejecutar su bloque por cada recoleccion que haga el Collect

~~Fljos~~
StateFlow

Hot: es caliente porque siempre mantiene un valor actual en memoria Aunque la UI cambie o se recomponga el ultimo valor o estado seguira disponible

Para estados UI se prefiere los StateFlow por su caracteristica que mantiene el ultimo estado y permite al compose que pueda actualizarse automaticamente cuando cambia el estado

I. 4 Inyección de dependencias (DI) Manual

La DI manual trata de la creación de dependencias que son principales dentro de una clase que extiende de Application

Ahi se crean la base de datos los DAO y repositorio para después las pantallas acceden a esas instancias mediante un constructor

```
val app = applicationContext as MiApplication
```

Por otro lado la palabra by lazy sirve para que un objeto se cree cuando se usa por primera vez y mantiene la misma instancia durante la ejecución del App

III. Desarrollo de código y lógica Personalizada

III.1 Repositorio y DI

```
Class Usuario Repository(  
    private val usuarioDAO : UsuarioDAO  
) {  
    obtener  
    fun obtener Usuarios() = usuarioDAO.obtener Usuarios()  
}
```

Usando Application y by lazy

```
Class MiApplication : Application() {  
    val  
    val database by lazy {  
        AppDatabase.getDatabase(this)  
    }  
  
    val usuarioRepository by lazy {  
        usuarioRepository(database.usuarioDAO())  
    }  
}
```

III.2 Logica Dinamica de Identidad

```
data class Usuario (  
    val nombre: String,  
    val apellido: String,  
    val edad: Int?  
)
```

```
fun main () {  
    val usuario: Usuario? = Usuario (  
        nombre = "Frank Cristian"  
        apellido = "Prado Lopez"  
        edad = null  
    )
```

```
    val puntaje = usuario?.let {  
        it.nombre.count { letra -> letra.isLetter() } +  
        it.apellido.count { letra -> letra.isLetter() }  
    } ?: 0
```

```
    val nombreCompleto = usuario?.let {  
        " ${ it.nombre } ${ it.apellido } "  
    } ?: "sin Nombre"
```

```
    Println ("Usuario: $nombreCompleto, tu puntaje de identidad es: $puntaje")
```

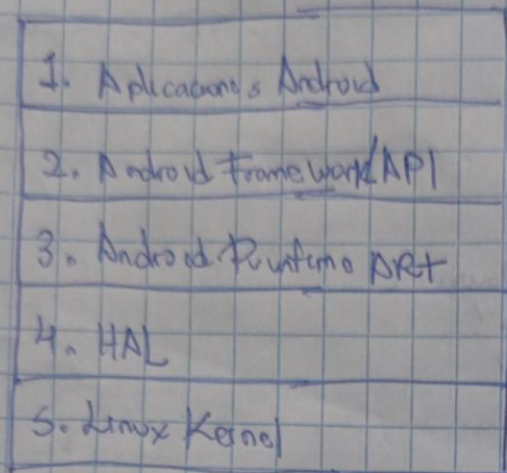
```
}
```

Resultado:

Usuario: Frank Cristian Prado Lopez, tu puntaje es: 23

IV: Arquitectura del AOSP Completa

IV.1 Gráfico



IV.2 Identificación total

1. Aplicaciones Android: Son las Apps instaladas por el usuario o el fabricante. Ejem: Whats App.
2. Android Framework / Api: Proporciona clases y servicios que usan las apps. En esta capa están los componentes como Activity Manager, etc.
3. Este ejecuta el código de las apps de Android convirtiendo el bytecode de la app en instrucciones que entiende el procesador.
Componentes
 - ART Runtime
 - Core Libraries
 - Garbage collector
 - Compilación AOT/JIT
 - Ejecución de Archivos DEX
4. HAL
Es el puente entre Android y los controladores del Hardware permite acceder a cámara, gps, sensores, audio, etc.
5. Linux Kernel: Núcleo del SO. Administra memoria, procesos, batería, seguridad y la comunicación con el Hardware.

II. Sección de Completer

1. El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo Flow
2. El operador withContext(Dispatchers.Default) permite ejecutar tareas pasadas fuera del main thread, específicamente optimizado para tareas de CPU como el parseo masivo de datos.
3. Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo android:foregroundServiceType = "location".
4. La función applicationContext de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast"
5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama SuspendCancellable Coroutine.
6. El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente AndroidManifest.xml
7. En la arquitectura ADSP, la capa que actúa como puente estándar entre el Framework y los controladores se denomina HAL o capa HAL
8. El operador de Flow que permite combinar múltiples fuentes (como GPS y Sensores) para emitir un estado unificado es combine()